Docker Lab: Flask API Containerization

Student: Yahya Houimdi,3A Info1

Exercise 1: Building and Running the Container

1.1 Dockerfile

1.2 Build the image:

bash
```
docker build -t flask-api .
```

1.3 Run the container:

bash
```
docker run -d -p 5000:5000 --name flask-api-container flask-api
```

1.4 Test the API:

bash
```
curl http://localhost:5000/books
```


```
{
  "books": [
    {
      "author": "George Orwell",
      "id": 1,
      "title": "1984",
      "year": 1949
    },
    {
      "author": "Harper Lee",
      "id": 2,
      "title": "To Kill a Mockingbird",
      "year": 1960
    },
    {
      "author": "F. Scott Fitzgerald",
      "id": 3,
      "title": "The Great Gatsby",
      "year": 1925
    },
```


1.5 docker logs fask-api-container

Exercise 2: API Testing

2.1 GET all books:

bash
```
curl -X GET http://localhost:5000/books
```

2.2 POST a new book:

bash
```
curl -X POST http://localhost:5000/books -H "Content-Type: application/json" -d '{"title":"Fahrenheit 451","author":"Ray Bradbury","year":1953}'
```

2.3 PUT (update) a book:

bash
```
curl -X PUT http://localhost:5000/books/1 -H "Content-Type: application/json" -d '{"title":"New World","author":"Aldous Huxley","year":1932}'
```

2.4 DELETE a book:

bash
```
curl -X DELETE http://localhost:5000/books/1
```

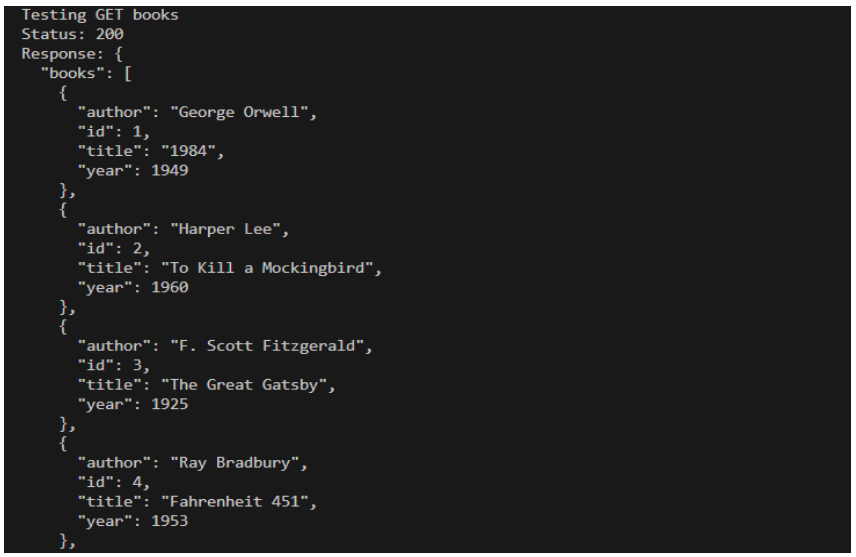
Output:

{"message": "Book deleted successfully"}

2.5 OK

2.6 Run the test script:

bash
```
python test_api.py
```



Exercise 3: Container Management

3.1 Stop the container:

bash
```
docker stop flask-api-container
```

Output: `flask-api-container`

3.2 Start the container:

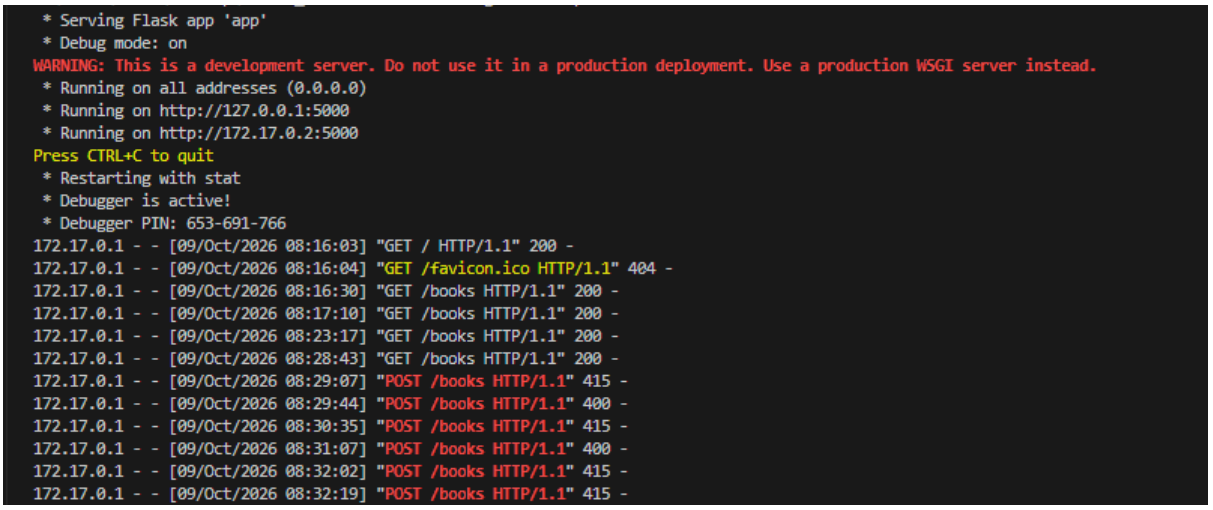
bash
```
docker start flask-api-container
```

Output: `flask-api-container`

3.3 View the container logs:

bash
```
docker logs flask-api-container
```

Output:



3.4 Check the Python version inside the container:

bash
```
docker exec -it flask-api-container python --version
```

Output: `Python 3.12.8`

Exercise 4: Security – Non-Root User

4.1 Dockerfile-apiuser

4.2 Build the image:

bash
```
docker build -f Dockerfile-apiuser -t flask-api-user .
```

4.3 Replace the running container with the non-root version:

bash
```
docker stop flask-api-container
docker rm flask-api-container
docker run -d -p 5000:5000 --name flask-api-user-container flask-api-user
```

4.4 Verify the user inside the container:

bash
```
docker exec -it flask-api-user-container whoami
```

Output: `apiuser`

Exercise 5: Log Management

5.1 Create a volume for the logs:

bash
```
docker volume create log-volume
```

Output: `log-volume`

5.2 Chosen approach: **(a) FileHandler in app.py**

Justification:

- Gives full control over the log format, level, and rotation.
- Follows Python best practices (`logging` module).
- Works independently of how the container is launched.
- With (a), a `RotatingFileHandler` limits file size, which is essential for a persistent volume that must not grow indefinitely.
- Approach (b) requires the shell form of `CMD`, which can interfere with signal propagation (SIGTERM) and prevent a clean server shutdown.

Modifications are in `app.py`.

5.3 Build the image with logging:

bash
```
docker build -f Dockerfile-logs -t flask-api-logs .
```

5.4 Generate logs

Success case:

bash
```
curl http://localhost:5000/books
curl -X POST http://localhost:5000/books -H "Content-Type: application/json" -d
'{"title":"Dune","author":"Frank Herbert","year":1965}'
```

Error case:

bash
```
curl -X PUT http://localhost:5000/books/999 -H "Content-Type: application/json" -d
'{"title":"Ghost"}'
```

5.5 Stop and remove the container:

bash
```
docker stop flask-api-logs-container
docker rm flask-api-logs-container
```

5.6 Read the logs from the volume (they persist after the container is removed):

bash
```
docker run --rm -v log-volume:/data alpine cat //data/api.log
```

5.7 Start a new container with the same volume and verify the logs persist:

bash
```
docker run -d -p 5000:5000 -v log-volume:/var/log/api --name flask-api-logs-container2
flask-api-logs
curl http://localhost:5000/books
docker run --rm -v log-volume:/data alpine cat //data/api.log
```